

CHL: memoria de agente local-first, MCP-native y aprendible

Arquitectura final de memoria semántica ligera para IA

Documento técnico de desarrollo
Adaptado al estado final del sistema

Mayo de 2026

Resumen

Este documento describe la versión final de CHL como una memoria de agente local-first, MCP-native, persistente y aprendible. La idea central sigue siendo usar firmas binarias sensibles a la localidad para recuperar vecinos semánticos, pero el sistema final ya no depende de un “hash” criptográfico como representación de significado. En su lugar, combina: normalización y canonicalización, un lexicón aprendido de conceptos y frases, firmas semánticas, hipervectores, persistencia en disco, journal incremental, backup binario comprimido y exposición por HTTP y MCP.

La implementación real se reparte entre `src/memory.js`, `src/native.js`, `native/chl_addon.cc`, `src/concepts.js`, `src/server.js` y `src/mcp.js`. El servidor HTTP expone inspección y exportación del estado y del lexicón; el servidor MCP expone herramientas y recursos para que un LLM lea, consulte y actúe sobre la memoria. Los benchmarks reproducibles en `scripts/evaluate.js` muestran dos regímenes de interés: en `bench:large` con 12 000 documentos y 6 000 consultas, `chl-native` alcanza `recall@1` de 0.9392 y p95 de 1255.92 μs ; en `bench:huge` con 50 000 documentos y 20 000 consultas, el `recall@1` global queda en 0.80925 y la latencia p95 en 863.54 μs , con una degradación esperable de paráfrasis al escalar. El resultado posiciona a CHL no como una base vectorial genérica, sino como una memoria conversacional viva para agentes: inspectable, exportable, persistente y con vocabulario aprendido entre sesiones.

Palabras clave: memoria de agente, MCP, LSH, SimHash, HDC, VSA, lexicón aprendido, distancia de Hamming, memoria semántica ligera.

Índice

1. Introducción	3
2. Estado final del sistema	3
2.1. Capas funcionales	3
2.2. Perfil real de ejecución	4
3. Fundamentos técnicos	4
3.1. Por qué no usar un hash criptográfico como semántica	4
3.2. Locality-Sensitive Hashing	4
3.3. SimHash	4
3.4. HDC y VSA	4

4. Arquitectura de memoria	5
4.1. Qué guarda cada entrada	5
4.2. Scoring final	5
4.3. Lexicón aprendido	5
4.4. Persistencia	5
5. Interfaz HTTP	6
6. Interfaz MCP	6
6.1. Herramientas expuestas	6
6.2. Recursos expuestos	7
7. Backup y restauración	7
8. Implementación nativa y fallback	8
9. Benchmarks reales	8
9.1. Benchmark large	8
9.2. Benchmark huge	9
9.3. Lectura de ingeniería	9
10.Comparación con alternativas de mercado	10
11.Limitaciones reales	10
12.Conclusión	10

1. Introducción

El objetivo de CHL no es sustituir a los Transformers ni competir como base de datos vectorial universal. El objetivo es más concreto: ofrecer a un agente una memoria local que pueda recordar, inspeccionar, exportar y reutilizar conocimiento con baja latencia y sin infraestructura pesada.

La motivación técnica es doble. Por un lado, los hashes criptográficos destruyen similitud: son excelentes para integridad, pero malos para semántica. Por otro lado, las memorias densas o vectoriales generalistas suelen ser potentes, pero no siempre son la solución más simple cuando lo que se necesita es memoria viva de agente, explicable y conectada al modelo por MCP.

CHL resuelve esa tensión con una arquitectura híbrida:

- firmas semánticas compactas para direccionamiento rápido;
- hipervectores para composición y re-ranking;
- lexicón aprendido para paráfrasis y frases recurrentes;
- persistencia local, journal y backup binario;
- API HTTP y MCP para inspección y control.

2. Estado final del sistema

La implementación final ya existe en el repositorio y se divide en módulos concretos:

Archivo	Responsabilidad real
src/memory.js	Motor JS de memoria asociativa, representaciones múltiples, scoring y fallback funcional.
native/chl_addon.cc	Addon nativo C++ con hash semántico, búsqueda rápida, persistencia y métricas.
src/native.js	Wrapper de alto nivel, fallback JS, persistencia de journal y sidecars del lexicón.
src/concepts.js	Canonicalización, lexicón de conceptos y frases, aprendizaje y serialización TSV.
src/server.js	API HTTP para estado, backup, restore, lexicón e inspección.
src/mcp.js	Herramientas y recursos MCP para memoria, backup, restore, lexicón y estado.
scripts/evaluate.js	Benchmarks reproducibles: small, large y huge.
test/chl.test.js	Tests de recall, persistencia, HTTP, MCP y lexicón.

Tabla 1: Mapa funcional del sistema final.

2.1. Capas funcionales

1. **Memoria local:** entradas, snapshot, journal, buckets, backup binario.
2. **Memoria aprendida:** conceptos y frases persistentes en `memory.concepts.tsv` y `memory.phrases.tsv`.
3. **Interfaz de agente:** herramientas y recursos MCP para acceso directo desde el LLM.
4. **Interfaz de inspección:** endpoints HTTP para consultar estado, entradas, backup y lexicón.

2.2. Perfil real de ejecución

El runtime soporta dos presets de producto:

- **default**: 128 bits, 32 band bits, 256 dimensiones hipervectoriales, 4096 entradas máximas.
- **large**: 128 bits, 32 band bits, 256 dimensiones hipervectoriales, 20 000 entradas máximas y 96 candidatos máximos.

El benchmark **bench:huge** no introduce un perfil nuevo de runtime: usa el preset **large** como estrés sobre un corpus de 50 000 documentos.

3. Fundamentos técnicos

3.1. Por qué no usar un hash criptográfico como semántica

Un hash criptográfico ofrece una propiedad opuesta a la que necesitamos: entradas parecidas producen salidas no correlacionadas. Eso es correcto para integridad, pero incorrecto para memoria semántica. En CHL, el hash útil debe preservar vecindad de forma probabilística.

3.2. Locality-Sensitive Hashing

Una familia $\mathcal{H}$ es sensible a localidad si entradas cercanas colisionan con mayor probabilidad que entradas lejanas:

$$\begin{aligned}\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] &\geq p_1 \quad \text{si } d(x, y) \leq r, \\ \Pr_{h \sim \mathcal{H}}[h(x) = h(y)] &\leq p_2 \quad \text{si } d(x, y) \geq cr,\end{aligned}$$

con $p_1 > p_2$ y $c > 1$. Esta es la base conceptual de la recuperación por buckets en CHL.

3.3. SimHash

En SimHash, cada bit se define por el signo de una proyección:

$$h_i(x) = \mathbb{I}[\langle r_i, \phi(x) \rangle \geq 0].$$

La distancia de Hamming se calcula con XOR y popcount:

$$d_H(a, b) = \text{popcnt}(a \oplus b).$$

La similitud binaria puede expresarse como:

$$\text{sim}_H(a, b) = 1 - \frac{d_H(a, b)}{m}.$$

3.4. HDC y VSA

HDC y VSA aportan las operaciones de composición:

- **binding**: asociación de roles y valores;
- **bundling**: superposición de múltiples señales;

- **permutación:** codificación de orden y contexto temporal;
- **cleanup:** limpieza por prototipo o memoria asociativa.

La implementación final no promete que un hash aislado “contenga conocimiento”; el conocimiento aparece al combinar firma, hipervector, payload, lexicón aprendido y memoria recuperable.

4. Arquitectura de memoria

4.1. Qué guarda cada entrada

Cada entrada almacenada por el sistema puede contener:

- texto original;
- representaciones normalizadas y canonicalizadas;
- firma semántica compacta;
- hipervector;
- payload recuperable;
- metadata como calidad, timestamps y contador de acceso;
- relaciones o información estructurada cuando el payload lo permita.

4.2. Scoring final

El ranking recupera candidatos a partir de la firma semántica y los reordena con señales de similitud, hipervector, concepto, recencia, calidad y negación. La forma conceptual del score es:

$$score(q, e) = \lambda_h sim_h + \lambda_v sim_v + \lambda_c sim_c + \lambda_q quality + \lambda_r recency + \lambda_n negation.$$

La implementación está repartida entre el fallback JS y el addon nativo para mantener comportamiento alineado.

4.3. Lexicón aprendido

El sistema final añadió una capa lateral de aprendizaje: un lexicón persistente de conceptos y frases. Esta capa vive en `src/concepts.js`, se escribe como TSV y se carga desde las variables de entorno:

- `CHL_CONCEPTS_PATH`
- `CHL_PHRASES_PATH`

En la práctica, esto permite que un LLM o una integración externa reutilice el vocabulario aprendido entre sesiones sin reconstruirlo desde cero.

4.4. Persistencia

La persistencia se resuelve en tres niveles:

1. **journal incremental**: eventos de **remember** y **learn**;
2. **backup binario**: archivo comprimido con cabecera CHLB y compresión **deflate**;
3. **sidecars del lexicón**: TSV para conceptos y frases.

5. Interfaz HTTP

El servidor HTTP expone operaciones de inspección, exportación y restauración. Los endpoints reales son:

Endpoint	Función
GET /health	Estado del servidor con snapshot.
GET /snapshot	Snapshot actual de la memoria.
GET /profile	Perfil activo (default o large).
GET /state	Estado completo, incluyendo entradas, bucket stats y journal.
GET /entries	Lista completa de entradas.
GET /journal	Journal de mutaciones.
GET /backup	Backup JSON compatible.
GET /backup.bin	Backup binario comprimido.
GET /lexicon	Lexicón aprendido en JSON con conteos y rutas.
GET /lexicon.concepts.tsv	Exportación manual de conceptos.
GET /lexicon.phrases.tsv	Exportación manual de frases.
GET /lexicon.tsv	Exportación combinada para inspección manual.
POST /remember	Inserta memoria.
POST /recall	Consulta vecinos.
POST /infer	Inferencia de mejor respuesta.
POST /learn	Aplica feedback.
POST /restore	Restaura backup JSON.
POST /restore.bin	Restaura backup binario.

Tabla 2: API HTTP real del sistema.

6. Interfaz MCP

El diseño MCP es central en el resultado final porque el agente puede acceder a la memoria como herramientas y recursos estándar. Esto encaja con la idea oficial de MCP: exponer recursos para aportar contexto y herramientas para ejecutar acciones controladas [8, 9].

6.1. Herramientas expuestas

Tool	Función
chl_remember	Guardar memoria.

<code>chl_recall</code>	Recuperar candidatos.
<code>chl_infer</code>	Inferir la mejor respuesta.
<code>chl_learn</code>	Aplicar feedback.
<code>chl_backup</code>	Exportar backup JSON.
<code>chl_backup_binary</code>	Exportar backup binario en base64.
<code>chl_restore</code>	Restaurar backup JSON.
<code>chl_restore_binary</code>	Restaurar backup binario.
<code>chl_snapshot</code>	Inspeccionar snapshot.
<code>chl_profile</code>	Consultar perfil activo.
<code>chl_state</code>	Ver estado completo.
<code>chl_entries</code>	Ver entradas completas.
<code>chl_journal</code>	Ver journal.
<code>chl_bucket_stats</code>	Ver estadísticas de buckets.
<code>chl_clear</code>	Vaciar memoria.
<code>chl_lexicon</code>	Inspeccionar lexicón aprendido.
<code>chl_lexicon_export</code>	Exportar lexicón en formato TSV para reutilización manual.

Tabla 3: Herramientas MCP reales de la implementación.

6.2. Recursos expuestos

Resource URI	Contenido
<code>chl://memory</code>	Estado completo de memoria.
<code>chl://profile</code>	Perfil activo.
<code>chl://state</code>	Estado, entradas, bucket stats y journal.
<code>chl://entries</code>	Entradas completas.
<code>chl://journal</code>	Journal.
<code>chl://backup</code>	Backup JSON.
<code>chl://backup.bin</code>	Backup binario codificado en base64.
<code>chl://lexicon</code>	Lexicón aprendido en JSON.
<code>chl://lexicon.concepts</code>	TSV de conceptos.
<code>chl://lexicon.phrases</code>	TSV de frases.
<code>chl://lexicon.tsv</code>	Exportación TSV combinada.

Tabla 4: Recursos MCP reales de la implementación.

7. Backup y restauración

El backup binario usa una estructura compacta con cabecera y compresión `deflate`. La restauración reconstruye la memoria y, si el backup contiene lexicón, vuelve a escribir los sidecars correspondientes. Esto permite reinicios limpios y reutilización entre sesiones.

La serialización de lexicón también es manualmente exportable en TSV para inspección, sincronización o despliegue en otra instancia.

8. Implementación nativa y fallback

El addon nativo (`native/chl_addon.cc`) implementa el camino de alto rendimiento. El wrapper `src/native.js` detecta si el binding está disponible y, si no, cae al motor JS. Esa decisión hace que el sistema sea portable durante desarrollo y de bajo riesgo durante despliegue.

```
1 class ChlEngine {
2 public:
3     uint32_t add(std::string_view text, Metadata meta);
4     std::vector<Result> search(std::string_view query, size_t k);
5     void reward(uint32_t query_id, uint32_t entry_id, float value);
6     void compact();
7 };
```

Listing 1: Esquema conceptual del motor en el addon nativo.

La implementación prioriza:

- popcount nativo;
- datos contiguos;
- menor asignación en el camino caliente;
- índices compactos;
- determinismo por semilla.

9. Benchmarks reales

Los benchmarks están definidos en `scripts/evaluate.js`. El modo `large` usa 12 000 documentos y 6 000 consultas. El modo `huge` usa 50 000 documentos y 20 000 consultas, pero sigue corriendo sobre el preset `large` del runtime. Los números siguientes son los últimos resultados reportados por el script.

9.1. Benchmark large

Métrica	chl-native	chl-js
docs	12 000	12 000
queries	6 000	6 000
learnedConcepts	16	16
learnedPhrases	18	18
loadMs	4 451.15	3 571.75
recall@1	0.93917	0.93917
recall@3	0.93983	0.93983
mrr	0.93950	0.93950
medianLatencyUs	452.21	453.90
p95LatencyUs	1 255.92	1 329.83
collisionBuckets	1 672	1 672
maxBucketSize	11	11

Tabla 5: Resultado del benchmark `bench:large`.

Categoría	recall@1
large-positive	0.93438
large-paraphrase	0.93286
large-negation	0.98351

Tabla 6: Desglose por categoría del benchmark large.

9.2. Benchmark huge

Métrica	chl-native	chl-js
docs	50 000	50 000
queries	20 000	20 000
learnedConcepts	60	60
learnedPhrases	66	66
loadMs	123 221.24	108 347.83
recall@1	0.80925	0.80905
recall@3	0.80960	0.80960
mrr	0.80943	0.80933
medianLatencyUs	292.75	316.77
p95LatencyUs	863.54	948.79
collisionBuckets	2 789	2 789
maxBucketSize	9	9

Tabla 7: Resultado del benchmark `bench:huge`.

Categoría	recall@1	Lectura
large-positive	0.83913	Buen comportamiento en consultas afirmativas directas.
large-paraphrase	0.76454	La paráfrasis sigue funcionando, pero ya muestra degradación al escalar.
large-negation	0.86859	La negación explícita se mantiene robusta.

Tabla 8: Desglose por categoría del benchmark huge.

9.3. Lectura de ingeniería

Los números de los benchmarks dejan cuatro conclusiones realistas:

1. La consulta es rápida incluso con decenas de miles de entradas.
2. La carga inicial es el cuello de botella principal en 50k.
3. La negación está bien modelada como señal explícita.
4. La paráfrasis es la zona más sensible al crecimiento del corpus.

10. Comparación con alternativas de mercado

CHL no pretende reemplazar a Weaviate, Pinecone o Redis como motor universal de retrieval. La comparación correcta es de caso de uso.

La clave es simple: si el problema es memoria conversacional local e inspeccionable, CHL tiene una ventaja clara. Si el problema es retrieval documental de gran escala, las plataformas generalistas siguen siendo superiores.

11. Limitaciones reales

- **No hay contexto infinito:** la memoria crece, pero el coste de carga y compactación existe.
- **La paráfrasis cae al escalar:** el benchmark de 50k muestra degradación frente a 12k.
- **El modelo no reemplaza comprensión profunda:** la codificación de características sigue siendo decisiva.
- **La similitud filtra información:** LSH no es privacidad.

12. Conclusión

El resultado final de CHL es una memoria de agente local-first, MCP-native, persistente, aprendible y explicable. El sistema ya no está en fase de idea: existe como implementación en C++ y JavaScript, con HTTP, MCP, backup binario, journal, sidecars del lexicón, tests y benchmarks.

La contribución técnica más importante es que el sistema separa tres cosas que suelen mezclarse:

- el direccionamiento semántico rápido por firmas binarias;
- la composición y limpieza por hipervectores;
- el vocabulario aprendido que persiste entre sesiones.

Los resultados experimentales son coherentes con esa lectura. En 12k, la paráfrasis supera 0.93 de recall@1; en 50k baja a 0.76, pero la consulta sigue siendo rápida y la negación se mantiene robusta. Eso sugiere que CHL ya es útil como memoria de agente, pero que el siguiente trabajo debe enfocarse en compactación, escalado del lexicón y preservación de paráfrasis sin aumentar ruido.

Apéndice: interfaz mínima para uso real

- HTTP: /lexicon, /lexicon.tsv, /backup.bin, /state.
- MCP tools: chl_remember, chl_recall, chl_infer, chl_lexicon, chl_backup_binary, chl_restore_binary.
- Recursos MCP: chl://memory, chl://lexicon, chl://backup.bin.

Referencias

- [1] P. Indyk y R. Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, 1998.

- [2] M. S. Charikar. Similarity estimation techniques from rounding algorithms. *Proceedings of the 34th Annual ACM Symposium on Theory of Computing*, 2002.
- [3] A. Z. Broder. On the resemblance and containment of documents. *Compression and Complexity of Sequences*, 1997.
- [4] P. Kanerva. *Sparse Distributed Memory*. MIT Press, 1988.
- [5] P. Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1(2):139–159, 2009.
- [6] T. A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995.
- [7] R. W. Gayler. Vector symbolic architectures answer Jackendoff’s challenges for cognitive neuroscience. *Joint International Conference on Cognitive Science*, 2003.
- [8] Model Context Protocol. Resources. Documentación oficial. <https://modelcontextprotocol.io/docs/concepts/resources>.
- [9] Model Context Protocol. Tools. Documentación oficial. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>.
- [10] Weaviate. Hybrid search. Documentación oficial. <https://docs.weaviate.io/weaviate/concepts/search/hybrid-search>.
- [11] Pinecone. Hybrid search. Documentación oficial. <https://docs.pinecone.io/guides/search/hybrid-search>.
- [12] Redis. Vector search concepts. Documentación oficial. <https://redis.io/docs/latest/develop/ai/search-and-query/vectors/>.

Criterio	CHL	Weaviate	Pinecone	Redis
Memoria de agente	Muy fuerte; conversacional y MCP-native	Search generalista	Retrieval de producción	Infraestructura de datos
MCP nativo	Sí	No	No	No
Explicabilidad	Alta: state, journal, backup, buckets y lexicón	Media	Media-baja	Media
Paráfrasis	Fuerte con lexicón aprendido	Buena con hybrid search	Buena con dense+sparse	Buena con vector + filtros
Negación	Señal explícita	Depende del ranking	Depende del esquema	Depende del índice
Local-first	Sí	Posible	Normalmente gestionado	Local o gestionado
Persistencia	Journal, backup y sidecars	Base de datos	Servicio	Infraestructura
Escala grande	Memoria local; no foco principal	Muy fuerte	Muy fuerte	Muy fuerte
Filtros/schema	Básico a medio	Muy fuerte	Fuerte	Muy fuerte
Coste operativo	Bajo	Medio/alto	Medio/alto	Medio
Mejor uso	Agente personal/producto	Búsqueda híbrida	RAG de producción	Search + cache + datos

Tabla 9: Comparación resumida con alternativas de mercado.